



Session 1: Introduction to Python, Jupyter Notebook and Basic Data Types

Compiled by D K. Muriithi

A Brief History of Python

Python was created by Guido van Rossum in the late 1980s and was first released in 1991. It was designed as a high-level, interpreted language with a focus on code readability and simplicity. The language takes inspiration from ABC, C, and Modula-3, aiming to provide a more approachable and efficient programming experience. Over the years, Python has evolved into one of the most widely used programming languages in data science, web development, automation, and artificial intelligence. Its vast ecosystem of libraries and strong community support make it a preferred choice for statistical analysis and machine learning.

What is Python?

Python is a versatile and widely used programming language in data analysis, machine learning, and statistical computing. It is popular due to its simplicity, readability, and vast ecosystem of libraries that support numerical computing, data manipulation, and visualization.

What is Jupyter Notebook?

Jupyter Notebook is an interactive web-based environment that allows users to write and execute Python code in a document-like format. It supports text, code execution, visualizations, and markdown documentation within a single file.

Installing Python and Jupyter Notebook

Python can be installed via:

Anaconda Distribution (Recommended): Comes with Python, Jupyter Notebook, and essential libraries pre-installed.

Standalone Python Installation: Using official Python downloads from python.org, followed by manual installation of Jupyter Notebook.

Steps to Install Using Anaconda:

Download and install Anaconda.

Open Anaconda Navigator and launch Jupyter Notebook.

Steps to Install Using pip:

Install Python from python.org.

Open the terminal or command prompt and run:

Method 1: Install PIP on Windows Using get-pip.py

The first method uses cURL to download the installation file and additional configuration steps post-installation. Follow the steps below to install PIP using this method:

Step 1: Download PIP get-pip.py

Before installing PIP, download the get-pip.py file. Run the following cURL command in the command prompt:

```
In [1]: #curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
```

Step 2: Installing PIP on Windows

To install PIP, run the following Python command:

```
In [2]: #python get-pip.py
```

Step 3: Verify Installation

To test whether the installation was successful, type the following command:

```
In [3]: #python -m pip help
```

Check PIP version installed

```
In [4]: #pip --version
```

Now install Jupyter Notebook and Jupyterlab

```
In [5]: #pip install jupyter
```

```
In [6]: #pip install notebook
```

```
In [7]: #pip install jupyterlab
```

Check the available jupyter version

```
In [8]: #jupyter --version
```

Jupyter Notebook Interface

Key components of Jupyter Notebook:

- Cells: Code and text (Markdown) can be written in separate cells.
- Kernel: The processing engine that executes code.
- Output: The results of executed code appear below the respective cell.

Basic Jupyter Notebook Commands

- Run a cell: Shift + Enter
- Insert a new cell: Esc + B
- Delete a cell: Esc + D + D
- Change cell type (Code/Markdown): Esc + M (for Markdown) or Esc + Y (for Code)

Basic Python Syntax and Data Types

Vectors

Vectors are one-dimensional data structures that store elements of the same type. They are commonly used in numerical computing and can represent mathematical quantities such as coordinates or time series data. In Python, vectors are typically implemented using lists or specialized numerical libraries such as NumPy.

Lists

Lists are ordered, mutable collections that can store elements of different types. They allow for dynamic resizing and provide various methods for element addition, removal, and manipulation. Lists are widely used in data storage, iteration, and flexible data handling.

Matrices

Matrices are two-dimensional structures that store data in rows and columns. They are essential in linear algebra, data transformations, and machine learning applications. Matrices facilitate mathematical operations such as addition, multiplication, and inversion. In Python, they are commonly handled using NumPy for efficient computation.

Tuples

Tuples are ordered collections similar to lists but are immutable, meaning their elements cannot be modified after creation. They are useful for storing fixed data, ensuring data integrity, and enhancing performance when iteration or access speed is critical.

Dictionaries

Dictionaries are data structures that store key-value pairs, allowing for fast data retrieval. They provide a flexible way to organize and access data based on unique keys. Dictionaries are widely used in applications requiring fast lookups, such as databases and data indexing.

Arrays

Arrays are homogeneous data structures that store elements of the same type in a contiguous memory location. They offer efficient storage and computation, making them suitable for numerical operations. Unlike lists, arrays require explicit type specification and are often used in scientific computing and large-scale data processing.

This is my first interaction with python (Basic Python Syntax and Codes)

Printing Output

```
In [9]: print("Hello, world")
```

Hello, world

```
In [10]: print(5+10)
```

15

Python Data types with syntax Examples

1. Integer: Represent whole numbers

```
In [11]: x = 20          # Integer
          x
          print(x)
```

20

2. Float: Represents decimal numbers

```
In [12]: y = 3.14      # Float  
print(y)
```

3.14

3. String: Represents a sequence of characters

```
In [13]: name = "Mercy" # String  
name
```

Out[13]: 'Mercy'

```
In [14]: name = 'Mercy'  
print(name)
```

Mercy

```
In [15]: greeting = 'Hello'  
greeting
```

Out[15]: 'Hello'

4. Boolean: Represents True or False values

```
In [16]: is_valid = True    # Boolean  
is_valid
```

Out[16]: True

```
In [17]: is_active = False # Boolean  
is_active
```

Out[17]: False

5. List: An ordered, mutable collection of items

```
In [18]: fruits = ["apple", "banana", "orange"]  
fruits
```

Out[18]: ['apple', 'banana', 'orange']

```
In [19]: fruits = ["apple", "banana", "orange"]  
print(fruits[0]) # Access first element  
print(fruits[1]) # Access second element  
print(fruits[2]) # Access third element
```

apple
banana
orange

6. Tuple: An ordered, immutable collection of items

```
In [20]: my_tuple = (1,2,3,"John", True)
my_tuple
```

```
Out[20]: (1, 2, 3, 'John', True)
```

```
In [21]: coordinates = (10, 20)
print(coordinates[1]) # Access second element
```

20

7. Set: An unordered mutable collection of unique items

```
In [22]: my_set = {1,2,3,"John"}
my_set
```

```
Out[22]: {1, 2, 3, 'John'}
```

8. Dictionary: A collection of key-value pairs

```
In [23]: my_dict = {
    "name" : "John",
    "Age": 60,
    "gender": "Male",
    "Occupation": "Doctor"}
my_dict
```

```
Out[23]: {'name': 'John', 'Age': 60, 'gender': 'Male', 'Occupation': 'Doctor'}
```

```
In [24]: person = {
    "Name": "John",
    "Age": 25,
    "Occupation": "Teacher"}
person
```

```
Out[24]: {'Name': 'John', 'Age': 25, 'Occupation': 'Teacher'}
```

```
In [25]: print(person["Name"]) # Access value by key
```

9. None Type: Represents the absence of a value

```
In [26]: x = None
```

10. What are Conditional Statements?

Conditional statements execute different code blocks based on whether a condition is True or False.

1. if Statement (do something if true)

```
In [27]: age = 18

if age >= 18:
    print("You are an adult")
```

You are an adult

2. if-else Statement (do one thing or another)

```
In [28]: age = 15

if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")
```

You are a minor

3. if-elif-else Statement (multiple conditions)

```
In [29]: age = 18
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Adult

```
In [30]: age = 10
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Minor

```
In [31]: age = 35
         if age >= 18:
             print("Adult")
         else:
             print("Minor")
```

Adult

```
In [32]: score = 85

         if score >= 90:
             print("Grade: A")
         elif score >= 80:
             print("Grade: B")
         elif score >= 70:
             print("Grade: C")
         else:
             print("Grade: F")
```

Grade: B

11. What are Loops?

Loops execute the same block of code repeatedly until a condition is met. There are two main types: for and while.

1. for Loop (repeat a fixed number of times)

Basic for loop

```
In [33]: # For Loop
         for i in range(5):
             print(i)
```

0
1
2
3
4

The code above defines a for loop that iterates over a sequence generated by the range(5) function. The range(5) function produces a sequence of numbers starting from 0 up to, but not including, 5, meaning it generates [0, 1, 2, 3, 4]. During each iteration, the variable i takes on the next value in this sequence, and the print(i) statement outputs the current value of i to the console. As a result, the numbers 0, 1, 2, 3, and 4 are printed sequentially. This type of loop is useful when iterating over a known range of values, such as indexing elements in a list or executing a block of code a fixed number of times.

Loop through a list


```
In [34]: fruits = ['apple', 'banana', 'cherry']

for fruit in fruits:
    print(fruit)
```

apple
banana
cherry

Loop through a string

```
In [35]: word = "Python"

for letter in word:
    print(letter)
```

P
y
t
h
o
n

while Loop (repeat while a condition is True)

```
In [36]: # While loop (repeat while a condition is True)
count = 0
while count < 3:
    print(count)
    count += 1
```

0
1
2

The code defines a while loop that continues executing as long as the condition `count < 3` is true. Initially, the variable `count` is set to 0. During each iteration, the current value of `count` is printed, and then `count` is incremented by 1 using `count += 1`. The loop runs until `count` reaches 3, at which point the condition `count < 3` becomes false, and the loop terminates. As a result, the numbers 0, 1, and 2 are printed sequentially. This type of loop is useful when the number of iterations is not predetermined but depends on a condition being met.

Range() Function (create a sequence of numbers)

Syntax	Description	Example
<code>range(5)</code>	0 to 4	<code>range(5)</code> → 0, 1, 2, 3, 4
<code>range(2, 5)</code>	2 to 4	<code>range(2, 5)</code> → 2, 3, 4
<code>range(0, 10, 2)</code>	0 to 9, step 2	<code>range(0, 10, 2)</code> → 0, 2, 4, 6, 8

```
In [37]: # Count from 1 to 5
for i in range(1, 6):
    print(i)
```

```
1
2
3
4
5
```

Loop Control Statements

```
In [38]: for i in range(10):
        if i == 5:
            break
        print(i)
```

```
0
1
2
3
4
```

Continue - Skip current iteration

```
In [39]: for i in range(5):
        if i == 2:
            continue
        print(i)
```

```
0
1
3
4
```

Pass - Do nothing (placeholder)

```
In [40]: for i in range(3):
        if i == 1:
            pass # Placeholder for future code
        print(i)
```

0
1
2

Nested Loops (loop inside a loop)

```
In [41]: for i in range(3):  
        for j in range(2):  
            print(f"i={i}, j={j}")
```

i=0, j=0
i=0, j=1
i=1, j=0
i=1, j=1
i=2, j=0
i=2, j=1

Print a multiplication table

```
In [42]: for i in range(1, 4):  
        for j in range(1, 4):  
            print(f"{i} × {j} = {i*j}")
```

1 × 1 = 1
1 × 2 = 2
1 × 3 = 3
2 × 1 = 2
2 × 2 = 4
2 × 3 = 6
3 × 1 = 3
3 × 2 = 6
3 × 3 = 9

Enumerate() - Get index and value

```
In [43]: fruits = ['apple', 'banana', 'cherry']  
  
        for index, fruit in enumerate(fruits):  
            print(f"{index}: {fruit}")
```

0: apple
1: banana
2: cherry

zip() - Loop through multiple lists

```
In [44]: names = ['Alice', 'Bob', 'Charlie']  
        scores = [85, 92, 78]  
  
        for name, score in zip(names, scores):  
            print(f"{name}: {score}")
```

Alice: 85
Bob: 92
Charlie: 78

With conditions

```
In [45]: even_numbers = [i for i in range(10) if i % 2 == 0]
         print(even_numbers)
```

[0, 2, 4, 6, 8]

Data Science Example - Loop through DataFrame

```
In [46]: import pandas as pd

         # Create sample data
         data = {
             'name': ['Alice', 'Bob', 'Charlie'],
             'score': [85, 62, 90]
         }

         df = pd.DataFrame(data)
         df
```

Out[46]:

	name	score
0	Alice	85
1	Bob	62
2	Charlie	90

```
In [47]: import pandas as pd

         # Create sample data
         data = {
             'name': ['Alice', 'Bob', 'Charlie'],
             'score': [85, 62, 90]
         }

         df = pd.DataFrame(data)

         # Loop through rows
         for index, row in df.iterrows():
             print(f"{row['name']} scored {row['score']}")
```

Alice scored 85
Bob scored 62
Charlie scored 90

12. Functions

```
In [48]: def greet(name):  
         return "Hello, " + name  
  
print(greet("Alice"))
```

Hello, Alice

13. Install Necessary libraries

```
In [49]: #!/pip install pandas  
#!/pip install numpy  
#!/pip install seaborn  
#!/pip install matplotlib  
#!/pip install wordcloud  
#!/pip install pillow  
#!/pip install plotly.express
```

14. Importing Libraries

```
In [50]: import pandas as pd           # Importing pandas for data manipulation  
import numpy as np                   # Importing NumPy for numerical computations  
import seaborn as sns               # Importing seaborn for data visualization  
import matplotlib.pyplot as plt     # Import the pyplot module
```

Mathematical Operations

1. Basic Arithmetic Operations

```
In [51]: x = 10  
y = 3  
  
addition = x + y           # Addition  
subtraction = x - y        # Subtraction  
multiplication = x * y      # Multiplication  
division = x / y           # Division (returns float)  
floor_division = x // y     # Floor division (integer division)  
modulus = x % y            # Modulus (remainder)  
exponentiation = x ** y     # Exponentiation (power)  
  
print(addition, subtraction, multiplication, division, floor_division, modulus, exp  
13 7 30 3.3333333333333335 3 1 1000
```

2. Using Built-in Math Functions

```
In [52]: import math  
  
square_root = math.sqrt(25) # Square root  
absolute_value = abs(8)      # Absolute value
```

```

power = pow(3, 3)           # 3 raised to the power of 3
logarithm = math.log(10)    # Natural Logarithm (ln)
log_base_10 = math.log10(100) # Logarithm base 10
factorial = math.factorial(5) # Factorial of 5

print(square_root, absolute_value, power, logarithm, log_base_10, factorial)

```

5.0 8 27 2.302585092994046 2.0 120

In [53]: `import math`

```

print(square_root)
print(absolute_value)
print(power)
print(logarithm)
print(log_base_10)
print(factorial)

```

5.0
8
27
2.302585092994046
2.0
120

3. Rounding Functions

In [54]: `import math`

```

num = 5.6789
rounded_value = round(num, 3) # Round to 2 decimal places
floor_value = math.floor(num) # Floor (round down)
ceil_value = math.ceil(num)   # Ceiling (round up)

print(rounded_value, floor_value, ceil_value)

```

5.679 5 6

Working with Vectors in Python

In [55]: `import numpy as np`

```

# Creating a vector
vector_a = np.array([1, 2, 3])
vector_a

```

Out[55]: `array([1, 2, 3])`

In [56]: `vector_b = np.array([4, 5, 6])`
`vector_b`

Out[56]: `array([4, 5, 6])`

Vector Addition

```
In [57]: vector_sum = vector_a + vector_b
vector_sum
```

```
Out[57]: array([5, 7, 9])
```

Vector Subtraction

```
In [58]: vector_diff = vector_a - vector_b
vector_diff
```

```
Out[58]: array([-3, -3, -3])
```

Scalar Multiplication

```
In [59]: scalar_multiplication = 3 * vector_a
scalar_multiplication
```

```
Out[59]: array([3, 6, 9])
```

Dot Product

```
In [60]: dot_product = np.dot(vector_a, vector_b)
dot_product
```

```
Out[60]: np.int64(32)
```

Magnitude of a Vector

```
In [61]: magnitude_a = np.linalg.norm(vector_a)
magnitude_a
rounded_magnitude_value = round(magnitude_a,3)
```

```
In [62]: print("Vector A:", vector_a)
print("Vector B:", vector_b)
print("Vector Sum:", vector_sum)
print("Vector Difference:", vector_diff)
print("Scalar Multiplication:", scalar_multiplication)
print("Dot Product:", dot_product)
print("Magnitude of Vector A:", rounded_magnitude_value)
```

```
Vector A: [1 2 3]
Vector B: [4 5 6]
Vector Sum: [5 7 9]
Vector Difference: [-3 -3 -3]
Scalar Multiplication: [3 6 9]
Dot Product: 32
Magnitude of Vector A: 3.742
```

Working with Lists in Python

Creating a list

```
In [63]: fruits = ["Apple", "Banana", "Cherry", "Date"]  
fruits
```

```
Out[63]: ['Apple', 'Banana', 'Cherry', 'Date']
```

Accessing elements

```
In [64]: first_fruit = fruits[0] # Access first element  
first_fruit
```

```
Out[64]: 'Apple'
```

```
In [65]: last_fruit = fruits[-1] # Access last element  
last_fruit
```

```
Out[65]: 'Date'
```

Modifying elements

```
In [66]: fruits[1] = "Blueberry" # Change Banana to Blueberry  
fruits
```

```
Out[66]: ['Apple', 'Blueberry', 'Cherry', 'Date']
```

Adding elements

```
In [67]: fruits.append("Elderberry") # Append new element to the end  
fruits.insert(2, "Coconut") # Insert at index 2
```

Removing elements

```
In [68]: deleted_fruit = fruits.pop(3) # Remove and return element at index 3  
fruits.remove("Date") # Remove first occurrence of "Date"  
fruits
```

```
Out[68]: ['Apple', 'Blueberry', 'Coconut', 'Elderberry']
```

List slicing

```
In [69]: subset_fruits = fruits[1:3] # Get elements from index 1 to 2  
subset_fruits
```

```
Out[69]: ['Blueberry', 'Coconut']
```


Iterating over a list

```
In [70]: for fruit in fruits:  
        print(fruit)
```

Apple
Blueberry
Coconut
Elderberry

Sorting a list

```
In [71]: fruits.sort() # Sort alphabetically  
fruits
```

Out[71]: ['Apple', 'Blueberry', 'Coconut', 'Elderberry']

Finding the length of a list

```
In [72]: num_fruits = len(fruits)  
num_fruits
```

Out[72]: 4

```
In [73]: print("Updated Fruits List:", fruits)  
print("Deleted Fruit:", deleted_fruit)  
print("Subset of Fruits:", subset_fruits)  
print("Number of Fruits:", num_fruits)
```

Updated Fruits List: ['Apple', 'Blueberry', 'Coconut', 'Elderberry']
Deleted Fruit: Cherry
Subset of Fruits: ['Blueberry', 'Coconut']
Number of Fruits: 4

MATRICES

What is a Matrix?

A matrix is a 2D array of numbers arranged in rows and columns.

○ Code



Matrix Example:

```
[1 2 3]  
[4 5 6]  
[7 8 9]
```

1. Creating Matrices with NumPy

Python



```
import numpy as np

# Method 1: Create from list
matrix1 = np.array([[1, 2, 3],
                    [4, 5, 6],
                    [7, 8, 9]])
print(matrix1)

# Method 2: Zeros matrix (all 0s)
zeros = np.zeros((3, 3))
print(zeros)

# Method 3: Ones matrix (all 1s)
ones = np.ones((2, 4))
print(ones)

# Method 4: Identity matrix (1s on diagonal)
identity = np.eye(3)
print(identity)

# Method 5: Random matrix
random_matrix = np.random.rand(3, 3)
print(random_matrix)

# Method 6: Range of numbers reshaped to matrix
matrix2 = np.arange(1, 10).reshape(3, 3)
print(matrix2)
```

2. Matrix Properties

Python



```
matrix = np.array([[1, 2, 3],
                  [4, 5, 6]])

print(matrix.shape)    # Output: (2, 3) - 2 rows, 3 columns
print(matrix.size)     # Output: 6 - total elements
print(matrix.ndim)     # Output: 2 - number of dimensions
print(matrix.dtype)    # Output: int64 - data type
```

3. Accessing Matrix Elements

Python



```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

# Single element
print(matrix[0, 0])    # Output: 1 (first row, first column)
print(matrix[1, 2])    # Output: 6 (second row, third column)

# Entire row
print(matrix[0])        # Output: [1 2 3]

# Entire column
print(matrix[:, 0])     # Output: [1 4 7]

# Slice (rows 0-1, columns 1-2)
print(matrix[0:2, 1:3]) # Output: [[2 3]
                          #         [5 6]]
```

4. Matrix Arithmetic Operations

Addition

Python



```
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

C = A + B
print(C)
# Output: [[ 6  8]
#         [10 12]]
```

Subtraction

Python



```
C = A - B
print(C)
# Output: [[-4 -4]
#         [-4 -4]]
```

Element-wise Multiplication

Python



```
C = A * B # Multiply corresponding elements
print(C)
# Output: [[ 5 12]
#          [21 32]]
```

Scalar Operations

Python



```
C = A * 2 # Multiply all elements by 2
print(C)
# Output: [[2 4]
#          [6 8]]

D = A + 10 # Add 10 to all elements
print(D)
# Output: [[11 12]
#          [13 14]]
```

5. Matrix Multiplication (Dot Product)

Python



```
A = np.array([[1, 2],
               [3, 4]])
B = np.array([[5, 6],
               [7, 8]])

# Matrix multiplication (linear algebra)
C = np.dot(A, B) # or A @ B
print(C)
# Output: [[19 22]
#          [43 50]]

# Explanation:
# [1*5 + 2*7, 1*6 + 2*8] = [19, 22]
# [3*5 + 4*7, 3*6 + 4*8] = [43, 50]
```

6. Transpose (flip rows and columns)

Python



```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

print(matrix.T)
# Output: [[1 4]
#          [2 5]
#          [3 6]]
```

7. Determinant and Inverse

Python



```
from numpy.linalg import det, inv

A = np.array([[1, 2],
              [3, 4]])

# Determinant
determinant = det(A)
print(f"Determinant: {determinant}") # Output: -2

# Inverse
A_inv = inv(A)
print(A_inv)
# Output: [[-2.  1.]
#          [ 1.5 -0.5]]

# Verify: A x A_inv = Identity matrix
print(np.dot(A, A_inv))
# Output: [[1. 0.]
#          [0. 1.]]
```

8. Eigenvalues and Eigenvectors

Python



```
from numpy.linalg import eig

A = np.array([[1, 2],
              [2, 3]])

eigenvalues, eigenvectors = eig(A)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

9. Matrix Operations Summary

Operation	Code	Description
Addition	<code>A + B</code>	Element-wise addition
Subtraction	<code>A - B</code>	Element-wise subtraction
Multiplication	<code>A * B</code>	Element-wise multiplication
Dot Product	<code>np.dot(A, B)</code> or <code>A @ B</code>	Matrix multiplication
Transpose	<code>A.T</code>	Flip rows and columns
Determinant	<code>np.linalg.det(A)</code>	Determinant value
Inverse	<code>np.linalg.inv(A)</code>	Matrix inverse
Rank	<code>np.linalg.matrix_rank(A)</code>	Rank of matrix
Trace	<code>np.trace(A)</code>	Sum of diagonal elements

Python



```
import numpy as np

# Sample data: 3 students, 4 subjects
scores = np.array([[85, 90, 78, 88],
                  [92, 88, 95, 91],
                  [78, 82, 85, 80]])

print("Student scores matrix:")
print(scores)
print(f"Shape: {scores.shape}")

# Average score per student
avg_per_student = scores.mean(axis=1)
print(f"\nAverage score per student: {avg_per_student}")

# Average score per subject
avg_per_subject = scores.mean(axis=0)
print(f"Average score per subject: {avg_per_subject}")

# Maximum score
print(f"Maximum score: {scores.max()}")

# Minimum score
print(f"Minimum score: {scores.min()}")

# Sum all scores
print(f"Total of all scores: {scores.sum()}")
```

Output:

○ Code



```
Student scores matrix:  
[[85 90 78 88]  
 [92 88 95 91]  
 [78 82 85 80]]  
Shape: (3, 4)  
  
Average score per student: [85.25 91.5  81.25]  
Average score per subject: [85.   86.67 86.   86.33]  
Maximum score: 95  
Minimum score: 78  
Total of all scores: 1008
```

11. Matrix with Pandas (Data Science)

● Python



```
import pandas as pd  
import numpy as np  
  
# Create DataFrame (matrix with labels)  
data = {  
    'Math': [85, 92, 78],  
    'Science': [90, 88, 82],  
    'English': [78, 95, 85]  
}  
  
df = pd.DataFrame(data, index=['Alice', 'Bob', 'Charlie'])  
print(df)  
  
# Access as matrix  
matrix = df.values  
print(type(matrix)) # Output: numpy.ndarray  
  
# Matrix operations  
print(df.mean())    # Average of each column  
print(df.sum())     # Sum of each column
```

12. Common Matrix Operations in Machine Learning

Python



```
# Feature scaling (normalize)
mean = np.mean(matrix, axis=0)
std = np.std(matrix, axis=0)
normalized = (matrix - mean) / std

# Linear combination for predictions
weights = np.array([0.5, 0.3, 0.2])
prediction = np.dot(features, weights)

# Covariance matrix (relationships between variables)
cov_matrix = np.cov(matrix.T)
```

Quick Practice Exercise:

Python



```
import numpy as np

# Create two 2x2 matrices
A = np.array([[1, 2],
               [3, 4]])
B = np.array([[2, 0],
               [1, 2]])

# Perform operations
print("A + B =\n", A + B)
print("\nA - B =\n", A - B)
print("\nA x B (element-wise) =\n", A * B)
print("\nA @ B (matrix multiplication) =\n", A @ B)
print("\nTranspose of A =\n", A.T)
```

Create DataFrame Using Pandas (Most Common)

In [74]: `import pandas as pd`

```
# Creating a DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", 'Dan'],
    "Age": [25, 30, 35, 40, 80],
    "County": ["Embu", "Nyeri", "Busia", "Bomet", 'Meru']
}
```



```
df = pd.DataFrame(data)
df
```

Out[74]:

	Name	Age	County
0	Alice	25	Embu
1	Bob	30	Nyeri
2	Charlie	35	Busia
3	David	40	Bomet
4	Dan	80	Meru

In [75]:

```
import pandas as pd
df = pd.DataFrame(data)
print(df.head())
#print(df.tail())
```

	Name	Age	County
0	Alice	25	Embu
1	Bob	30	Nyeri
2	Charlie	35	Busia
3	David	40	Bomet
4	Dan	80	Meru

Accessing Columns

In [76]:

```
df["County"]
```

Out[76]:

0	Embu
1	Nyeri
2	Busia
3	Bomet
4	Meru

Name: County, dtype: object

Accessing Rows

In [77]:

```
print("\nAccessing first row:")
print(df.iloc[0])
```

Accessing first row:

Name	Alice
Age	25
County	Embu

Name: 0, dtype: object

Adding a New Column

In [78]:

```
df["Salary"] = [50000, 60000, 70000, 80000, 90000]
df['occupation'] = ['Nurse', 'Teacher', 'Politician', 'Doctor', 'Nurse' ]
df
```

```
Out[78]:
```

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000	Nurse
1	Bob	30	Nyeri	60000	Teacher
2	Charlie	35	Busia	70000	Politician
3	David	40	Bomet	80000	Doctor
4	Dan	80	Meru	90000	Nurse

Filtering Data

```
In [79]: print("\nFiltering rows where Age > 30:")  
df[df["Age"] > 30]
```

Filtering rows where Age > 30:

```
Out[79]:
```

	Name	Age	County	Salary	occupation
2	Charlie	35	Busia	70000	Politician
3	David	40	Bomet	80000	Doctor
4	Dan	80	Meru	90000	Nurse

Sorting Data

```
In [80]: df.sort_values(by="Age", ascending=False)
```

```
Out[80]:
```

	Name	Age	County	Salary	occupation
4	Dan	80	Meru	90000	Nurse
3	David	40	Bomet	80000	Doctor
2	Charlie	35	Busia	70000	Politician
1	Bob	30	Nyeri	60000	Teacher
0	Alice	25	Embu	50000	Nurse

```
In [81]: df.sort_values(by="Age", ascending=True)
```

```
Out[81]:
```

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000	Nurse
1	Bob	30	Nyeri	60000	Teacher
2	Charlie	35	Busia	70000	Politician
3	David	40	Bomet	80000	Doctor
4	Dan	80	Meru	90000	Nurse

Grouping Data

```
In [82]: df.groupby("County")["Age"].mean()
```

```
Out[82]: County
Bomet      40.0
Busia      35.0
Embu       25.0
Meru       80.0
Nyeri      30.0
Name: Age, dtype: float64
```

Handling Missing Data

```
In [83]: df.loc[2, "Salary"] = None # Introduce a missing value (numerical)
df
```

```
Out[83]:
```

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000.0	Nurse
1	Bob	30	Nyeri	60000.0	Teacher
2	Charlie	35	Busia	NaN	Politician
3	David	40	Bomet	80000.0	Doctor
4	Dan	80	Meru	90000.0	Nurse

```
In [84]: print("\nDataFrame with missing value:")
df
```

```
DataFrame with missing value:
```

Out[84]:

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000.0	Nurse
1	Bob	30	Nyeri	60000.0	Teacher
2	Charlie	35	Busia	NaN	Politician
3	David	40	Bomet	80000.0	Doctor
4	Dan	80	Meru	90000.0	Nurse

In [85]:

```
print("\nFilling missing values with mean salary:")
df["Salary"].fillna(df["Salary"].mean(), inplace=True)
df
```

Filling missing values with mean salary:

C:\Users\Admin\AppData\Local\Temp\ipykernel_6228\2603864996.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["Salary"].fillna(df["Salary"].mean(), inplace=True)
```

Out[85]:

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000.0	Nurse
1	Bob	30	Nyeri	60000.0	Teacher
2	Charlie	35	Busia	70000.0	Politician
3	David	40	Bomet	80000.0	Doctor
4	Dan	80	Meru	90000.0	Nurse

Suppress Warnings

In [86]:

```
import warnings

# Suppress all warnings
warnings.simplefilter("ignore")

print("\nFilling missing values with mean salary:")
df["Salary"].fillna(df["Salary"].mean(), inplace=True)
df
```

Filling missing values with mean salary:

Out[86]:

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000.0	Nurse
1	Bob	30	Nyeri	60000.0	Teacher
2	Charlie	35	Busia	70000.0	Politician
3	David	40	Bomet	80000.0	Doctor
4	Dan	80	Meru	90000.0	Nurse

In [87]: `df.loc[2, "occupation"] = None # Introduce a missing value (Categorical)`
`df`

Out[87]:

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000.0	Nurse
1	Bob	30	Nyeri	60000.0	Teacher
2	Charlie	35	Busia	70000.0	None
3	David	40	Bomet	80000.0	Doctor
4	Dan	80	Meru	90000.0	Nurse

In [88]: `import warnings`
`# Suppress all warnings`
`warnings.simplefilter("ignore")`
`df["occupation"].fillna(df["occupation"].mode()[0], inplace=True) # Replace the mis`
`df`

Out[88]:

	Name	Age	County	Salary	occupation
0	Alice	25	Embu	50000.0	Nurse
1	Bob	30	Nyeri	60000.0	Teacher
2	Charlie	35	Busia	70000.0	Nurse
3	David	40	Bomet	80000.0	Doctor
4	Dan	80	Meru	90000.0	Nurse

Exercise: Create dataframe as guided below

```

import pandas as pd

# Method 1: From a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 75000]
}
df = pd.DataFrame(data)
print(df)

# Method 2: From a list of lists
data = [
    ['Alice', 25, 50000],
    ['Bob', 30, 60000],
    ['Charlie', 35, 75000]
]
df = pd.DataFrame(data, columns=['Name', 'Age', 'Salary'])
print(df)

# Method 3: From a CSV file
df = pd.read_csv('data.csv')
print(df)

# Method 4: From a NumPy array
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
df = pd.DataFrame(arr, columns=['A', 'B', 'C'])
print(df)

# Method 5: From a dictionary with lists (with index)
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}
df = pd.DataFrame(data, index=['Row1', 'Row2', 'Row3'])
print(df)

```

```

In [89]: # Exporting Data to CSV
df.to_csv("output.csv", index=False)

```



CHUKA UNIVERSITY
Knowledge is Wealth! Akili ni Mali
(Sapientia Divitia Est)

**CENTER FOR DATA ANALYTICS
AND MODELING**

ARTIFICIAL INTELLIGENCE | DATA SCIENCE | CONSULTANCY | RESEARCH HUB



SERVICES OFFERED

- Short Course Training in Data Science
- Research (Leveraging AI Tools)
- Consultancy

SHORT COURSE

- R & Rstudio
- Python
- Power BI
- STATA, SPSS & others

Phone : 020-2310512/18
Email: info@chuka.ac.ke

[ChukaUniversityOfficialPage](#) [ChukaUniversityOfficial](#) [@chuka_uni](#)
[Chuka University TV](#) [Chuka_University_Official](#) www.chuka.ac.ke

<https://cdam.chuka.ac.ke>

Thank you

